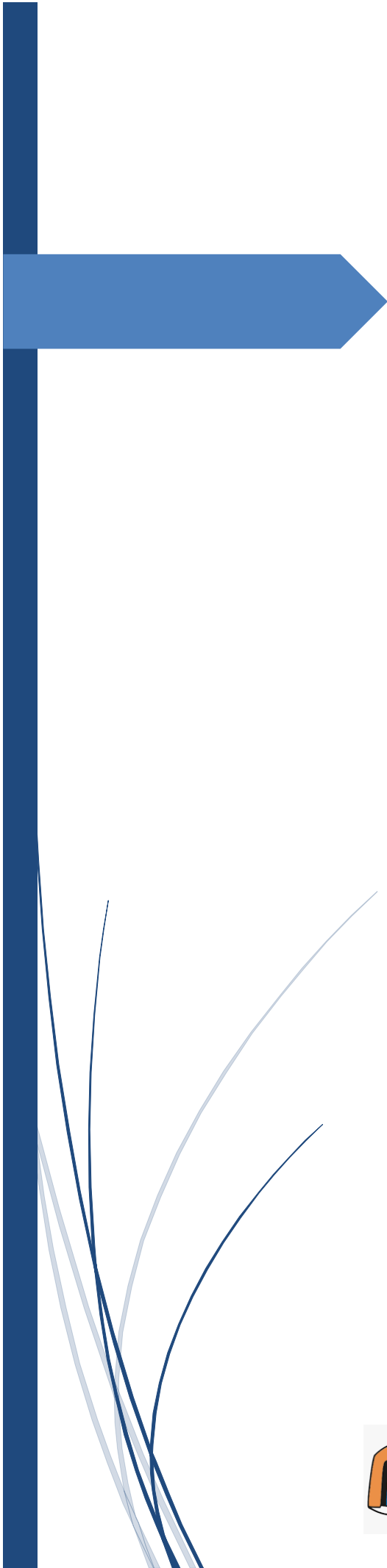




Memoria Practica 3

Bases de Datos 2

Álvaro Pérez Gregorio 870097
Mario Recio Lueza 838583



Escuela Universitaria
Politécnica - Teruel

Universidad Zaragoza

Indice:

Indice:.....	1
Introducción:.....	2
Objetivo:	2
Motivación:	2
1 y 2. Crea un ring con al menos 3 nodos y Realiza el diseño de la base de datos que se implementó en el módulo de SQL para Cassandra teniendo en cuenta el modelo de datos de Cassandra.....	3
3. Realiza un script de automatización de inserción de distintos volúmenes de datos (small, medium, large) y compáralo frente a la base de datos MySQL y MongoDB.	3
4. Realiza las mismas consultas que se realizaron en la práctica de SQL y en MongoDB (si no tienes ninguna consulta que involucre varias tablas, al menos 2-4 de ellas, añádela) y haz una comparativa del rendimiento.....	5
5. Investiga en qué te beneficia Cassandra frente al resto de bases de datos vistas previamente.....	6
• ¿Qué pasaría si un nodo dejará de estar disponible en un momento dado?	6
• ¿Podrías ser menos restrictivo en cuanto a la replicación de ciertos keyspaces y en que te beneficiaría?.....	6
• ¿Qué impacto tiene el tamaño del batch en las inserciones en Cassandra? ¿Es recomendable usarlos siempre? ¿Por qué?	6
• ¿Cómo podrías aprovechar la replicación y la configuración del Consistency Level para mejorar el rendimiento de las inserciones y consultas?	7
• ¿Qué estrategias ofrece Cassandra para particionar la información y distribuirla eficientemente en el ring? ¿Qué es un partition key y cómo afecta al rendimiento?	7
• ¿De qué forma influye la compactación y el memtable flushing en el rendimiento de Cassandra?	7
6. Los sistemas tradicionales como los RDBMS siguen las propiedades ACID. ¿Qué modelo de consistencia ofrece Cassandra? ¿Qué significa esto?	8
• ¿Existe duplicidad de información en tu modelo de Cassandra? ¿Es algo habitual en este tipo de bases de datos? ¿Por qué?	8
• ¿Cómo puedes evitar la duplicación de datos innecesaria tanto a nivel de base de datos como a nivel de aplicación, teniendo en cuenta que Cassandra favorece la denormalización?	8
• ¿Qué problemas puede provocar un mal diseño de las claves de partición respecto a la consistencia y disponibilidad de los datos?	9
7. Conclusiones obtenidas: ¿Crees que tu modelo de datos encaja mejor en un RDBMS o se adapta mejor a Cassandra?	10
• ¿Qué cambios has tenido que hacer respecto a un diseño relacional tradicional? ¿Cómo te ha condicionado pensar en las consultas desde el principio?	10
• ¿Qué ventajas e inconvenientes has observado en el uso de Cassandra para el caso de uso planteado?.....	10
8. Conclusiones finales	11
9. Anexo	12

Introducción:

En esta tercera práctica se nos propone desarrollar de una base de datos en Cassandra, un sistema gestor NoSQL basado en el modelo wide-column, partiendo de la temática que se desarrolló en el bloque de SQL y de MongoDB, se quiere transformar dicho modelo de base de datos adaptado al formato y particularidades de Cassandra teniendo en cuenta su estructura de almacenamiento por columnas, su modelo distribuido y su modelo de consultas basado en particiones.

Al igual que en prácticas anteriores, se realizará un análisis de rendimiento mediante la inserción automatizada de distintos volúmenes de datos (small, medium, large), y se ejecutarán las mismas consultas realizadas anteriormente para permitir una comparativa de eficiencia frente a MySQL y MongoDB.

Objetivo:

El principal objetivo de esta práctica es seguir profundizando en el uso de NoSQL, en este caso enfocándonos en un enfoque basado en almacenamiento por columnas con Cassandra.

Además de diseñar la base de datos funcional, se busca comparar su rendimiento en tareas de inserción y consulta frente a las prácticas anteriores, con el fin de entender las diferencias entre las bases de datos, relacionales, documentales y wide-column para finalmente elegir cual nos vendría mejor para nuestro propio diseño en cuanto a rendimiento y usabilidad.

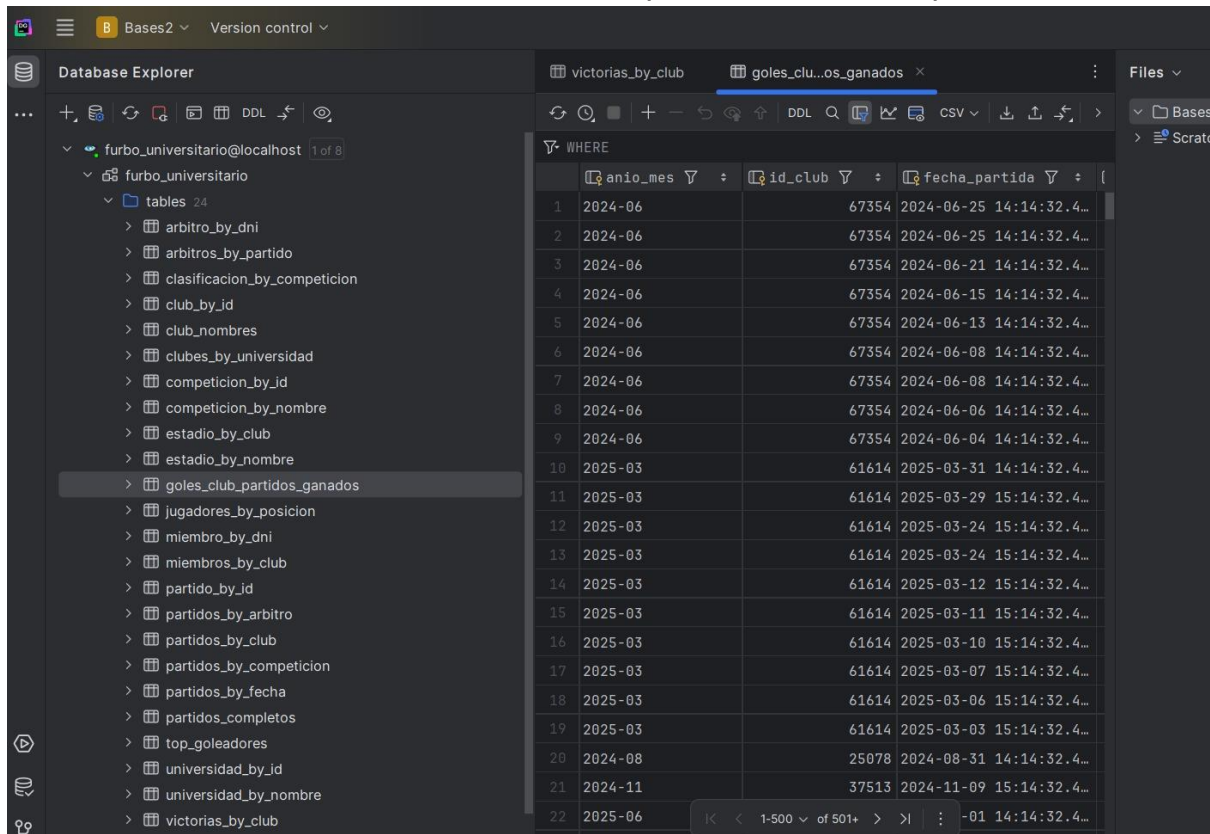
Motivación:

La realización de esta práctica nos da otra oportunidad de conocer un nuevo modelo de base de datos orientado al rendimiento en grandes volúmenes de datos y entornos distribuidos.

La comparación con MySQL y MongoDB nos permitirá no solo comprender cómo se modelan los datos en Cassandra, sino también valorar cuándo es más adecuado utilizar una base de datos wide-column. Así podremos ampliar nuestro abanico de posibilidades a la hora de realizar una base de datos en un futuro.

1 y 2. Crea un ring con al menos 3 nodos y Realiza el diseño de la base de datos que se implementó en el módulo de SQL para Cassandra teniendo en cuenta el modelo de datos de Cassandra

Aquí mostramos como quedan las tablas en la base de datos de nuestro tema “Fútbol Universitario” como venimos haciendo en las prácticas anteriores pero en Cassandra.



The screenshot shows the Database Explorer interface. On the left, the tree view shows the database 'furbo_universitario' with a list of tables. The table 'goles_club_partidos_ganados' is selected. On the right, the table view displays the data for this table. The columns are 'anio_mes', 'id_club', and 'fecha_partida'. The data is organized into rows, with the first 22 rows visible. The table has a primary key on 'anio_mes' and 'id_club'.

	anio_mes	id_club	fecha_partida
1	2024-06	67354	2024-06-25 14:14:32.4...
2	2024-06	67354	2024-06-25 14:14:32.4...
3	2024-06	67354	2024-06-21 14:14:32.4...
4	2024-06	67354	2024-06-15 14:14:32.4...
5	2024-06	67354	2024-06-13 14:14:32.4...
6	2024-06	67354	2024-06-08 14:14:32.4...
7	2024-06	67354	2024-06-08 14:14:32.4...
8	2024-06	67354	2024-06-06 14:14:32.4...
9	2024-06	67354	2024-06-04 14:14:32.4...
10	2025-03	61614	2025-03-31 14:14:32.4...
11	2025-03	61614	2025-03-29 15:14:32.4...
12	2025-03	61614	2025-03-24 15:14:32.4...
13	2025-03	61614	2025-03-24 15:14:32.4...
14	2025-03	61614	2025-03-12 15:14:32.4...
15	2025-03	61614	2025-03-11 15:14:32.4...
16	2025-03	61614	2025-03-10 15:14:32.4...
17	2025-03	61614	2025-03-07 15:14:32.4...
18	2025-03	61614	2025-03-06 15:14:32.4...
19	2025-03	61614	2025-03-03 15:14:32.4...
20	2024-08	25078	2024-08-31 14:14:32.4...
21	2024-11	37513	2024-11-09 15:14:32.4...
22	2025-06		-01 14:14:32.4...

3. Realiza un script de automatización de inserción de distintos volúmenes de datos (small, medium, large) y compáralo frente a la base de datos MySQL y MongoDB.

Para automatizar la inserción de datos hemos optado por hacer un script en javascript ya que con python nos daba muchos problemas, en dicho script hemos puesto tres opciones de inserción: small, medium y large, cada una con una cantidad de datos determinable.

Además de ello el script tiene dos modos: minimal(tan solo inserta datos sobre las tablas necesarias para las consultas) y full(inserta datos en todas las tablas), esto lo hemos diseñado de esta forma porque si insertamos en todas las tablas el tiempo de inserción es muy alto y no estaríamos usando todos los datos insertados, si lo hacemos insertando tan solo en las tablas necesarias el tiempo de inserción será mucho más rápido y tan solo usaremos los datos insertados. Para que la inserción sea lo más óptima posible se ha empleado el batch, esto lo que hace es que en vez de hacer las inserciones 1 a 1, se hacen x a x; en el caso de javascript el rango permitido es de 100-150.

También hemos empleado la librería faker para la generación de los datos, lo que nos ha facilitado mucho el trabajo.

A continuación se muestran los resultados obtenidos de las inserciones en modo minimal(en full no mostramos los resultados ya que eran muy altos, tan solo hicimos con el SMALL):

SMALL

Performance metrics (minimal mode):

- Data generation time: 3.23 seconds
- Database insertion time: 19.76 seconds
- Total items: 184980
- Insertion speed: 9362.76 items/second
- Tiempo(modos full): 49.34s

MEDIUM

Performance metrics (minimal mode):

- Data generation time: 23.79 seconds
- Database insertion time: 77.36 seconds
- Total items: 891010
- Insertion speed: 11517.11 items/second

LARGE

Performance metrics (minimal mode):

- Data generation time: 209.65 seconds
- Database insertion time: 236.82 seconds
- Total items: 2825820
- Insertion speed: 11932.45 items/second

	Small dataset	Medium dataset	Large dataset
MongoDB	0.971 seg	8.323 seg	65.825 seg
≈data insertada	147.086	922.124	4.178.143
MariaDB	1.342 seg	13.366 seg	304.268 seg
≈data insertada	160.850	943.000	3.816.000

Esta es la comparación que realizamos previamente para comparar MongoDB y MariaDB, con los datos obtenidos al insertar en Cassandra hemos visto reflejado que Cassandra es más lento que ambos motores, esto se debe a que estamos generando e insertando muchos datos para tablas muy extensas ya que estas contiene más variables de cara a luego poder realizar las consultas.

4. Realiza las mismas consultas que se realizaron en la práctica de SQL y en MongoDB (si no tienes ninguna consulta que involucre varias tablas, al menos 2-4 de ellas, añádela) y haz una comparativa del rendimiento.

CONSULTA 1- Goles de clubes en partidos ganados en un mes en específico(2025-04)

```
SELECT anio_mes, id_club, nombre_club, goles_ anotados  
FROM goles_club_partidos_ganados  
WHERE anio_mes = '2025-04' AND id_club IN (1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
```

CONSULTA 2- Todos los partidos de un equipo en específico poniendo su id

```
SELECT fecha_partida, id_partido, es_local,  
       nombre_rival, num_goles_favor, num_goles_contra  
FROM partidos_by_club  
WHERE id_club = 4004  
ORDER BY fecha_partida DESC;
```

CONSULTA 3 - Cuenta el total de victorias de todos los clubes

```
SELECT id_club, total_victorias  
FROM victorias_by_club;
```

CONSULTA 4 - Muestra los detalles de todos los partidos

```
SELECT id_partido, fecha_partida, nombre_competicion,  
       nombre_local AS equipo_local,  
       nombre_visitante AS equipo_visitante,  
       num_goles_local, num_goles_visitante,  
       arbitros_info AS arbitros  
FROM partidos_completos;
```

Las consultas son más o menos parecidas a las que hemos hecho en las otras prácticas, lo que cambia es el tiempo de ejecución. Estas consultas no duran apenas tiempo por lo que no hemos medido el tiempo ya que no llegan ni al segundo, esto se debe a que la base de datos está diseñada directamente para realizar estas consultas ya que en Cassandra no existen los JOIN por lo que hay que hacer el diseño de tal forma que en las consultas todos los datos que se requieran se puedan obtener de una tabla.

Con esta información queda más que claro que Cassandra es más óptima que MySQL y MariaDB a la hora de las consultas.

5. Investiga en qué te beneficia Cassandra frente al resto de bases de datos vistas previamente.

Cassandra es mejor frente a MongoDB y MariaDB si priorizamos la escalabilidad y el rendimiento sobre la complejidad del modelo de datos. Si se necesita soportar fallos y seguir funcionando es la mejor solución ya que tiene una alta tolerancia a fallos sin necesidad de configuraciones complejas.

Cuando vayamos a trabajar con datos altamente distribuidos geográficamente o con muchas escrituras por segundo también puede ser una mejor opción frente a las otras dos.

A continuación, contestaremos las preguntas que se nos presentan sobre este tema:

- **¿Qué pasaría si un nodo dejará de estar disponible en un momento dado?**

Por suerte no pasaría nada muy problemático ya que Cassandra tiene una particularidad y es que detecta automáticamente nodos caídos y redirige las peticiones a réplicas disponibles, de hecho si hay suficientes el sistema seguirá funcionando sin pérdida de datos.

Además cuando el nodo se reincorpore se sincronizará automáticamente.

- **¿Podrías ser menos restrictivo en cuanto a la replicación de ciertos keyspaces y en que te beneficiaría?**

Si que se podría ser más restrictivo, se podría definir el factor de replicación por keyspace.

Que sea menos restrictivo, es decir que tenga menos réplicas, si tiene beneficios tales como usar menos el disco o un mayor rendimiento en escrituras ya que hay menos nodos que escribir al mismo tiempo.

Pero no es todo tan bonito, habiendo beneficios también hay inconvenientes, y el más importante es que pierdes mucha tolerancia de fallos si solo cuentas con una réplica así que tienes que tener cuidado.

- **¿Qué impacto tiene el tamaño del batch en las inserciones en Cassandra? ¿Es recomendable usarlos siempre? ¿Por qué?**

Cassandra no utiliza los “batches” para transacciones como en SQL, si no que los usa para la optimización de la red. Si haces un “batch” grande con distintas particiones, puedes ralentizar el sistema hasta causar tiempos de espera conocidos como “timeouts”.

Contestando a si es recomendable usarlos siempre la respuesta es no.

Estaría bien usarlos para insertar múltiples filas de la misma partición pero habría que evitarlos para muchas particiones diferentes ya que no es eficiente.

- **¿Cómo podrías aprovechar la replicación y la configuración del Consistency Level para mejorar el rendimiento de las inserciones y consultas?**

La replicación permite que los datos se almacenen en varios nodos, lo que proporciona tolerancia a fallos y disponibilidad. Por otro lado, el Consistency Level define cuántos nodos deben responder a una operación de lectura o escritura para considerarla válida.

Para priorizar el rendimiento, por ejemplo, se pueden realizar inserciones con un Consistency Level como *ONE* ya que solo un nodo necesita confirmar la operación.

Combinando una estrategia de replicación adecuada con niveles de consistencia ajustados, se puede lograr un equilibrio entre rendimiento, disponibilidad y consistencia, optimizando tanto las escrituras como las consultas en Cassandra.

- **¿Qué estrategias ofrece Cassandra para particionar la información y distribuirla eficientemente en el ring? ¿Qué es un partition key y cómo afecta al rendimiento?**

Cassandra utiliza *partition keys* y un *hashing distribuido* el cual está por defecto el “Murmur3”.

El sistema se llama ring donde Cassandra balancea los datos automáticamente y cada nodo es responsable de un rango de particiones.

Por otro lado, la *partition key* determina en qué nodo se guarda cada dato y su rendimiento afecta directamente proporcional por lo que si hay un buen diseño la distribución será uniforme pero si hay alguna key muy repetida un solo nodo hará todo el trabajo por lo que será malo para el rendimiento.

- **¿De qué forma influye la compactación y el memtable flushing en el rendimiento de Cassandra?**

En la Compactación se combinan varios “SSTables” (archivos donde se guarda la info ordenada para poder leer rapido) en uno para reducir la fragmentación. El Memtable flushing sirve para cuando se llena la memoria se escribe en disco como un “SSTable”.

Sabiendo esto el impacto que tiene podría parecer muy bueno ya que se mejoran las lecturas al reducir la cantidad de archivos en disco, pero consume mucho CPU y disco al compactar, por eso se programa de esta forma cuidadosamente.

6. Los sistemas tradicionales como los RDBMS siguen las propiedades ACID. ¿Qué modelo de consistencia ofrece Cassandra? ¿Qué significa esto?

Cassandra no sigue el modelo *ACID* tradicional como los *RDBMS*, sino que adopta un modelo de consistencia llamado AP del teorema CAP, el cual prioriza la alta disponibilidad y tolerancia a particiones por encima de la consistencia estricta.

Su consistencia es ajustable, gracias al parámetro llamado *Consistency Level*, explicado en uno de los apartados anteriores, que permite al desarrollador elegir cuántos nodos deben de completar una operación para que se considere válida.

Esto significa que podemos optimizar Cassandra según nuestras necesidades: si buscamos rendimiento, usaremos niveles bajos (ONE o TWO), y si necesitamos mayor fiabilidad en los datos, niveles como QUORUM o ALL.

- **¿Existe duplicidad de información en tu modelo de Cassandra? ¿Es algo habitual en este tipo de bases de datos? ¿Por qué?**

De hecho si, es habitual encontrar duplicidad de información en modelos de Cassandra. A diferencia de los *RDBMS*, Cassandra fomenta la denormalización, lo que implica repetir datos en distintas tablas para mejorar el rendimiento de las consultas.

Esto se debe a que en Cassandra las consultas están muy ligadas al diseño del modelo de datos. Como no existen relaciones entre tablas ni *JOINS*, es común duplicar datos para evitar operaciones costosas y lograr una lectura eficiente.

- **¿Cómo puedes evitar la duplicación de datos innecesaria tanto a nivel de base de datos como a nivel de aplicación, teniendo en cuenta que Cassandra favorece la denormalización?**

A nivel de base de datos se podría evitar diseñando las tablas basándose en las consultas reales que se vayan a realizar y usando claves de partición bien pensadas para evitar repetir los mismos datos en múltiples particiones de manera innecesaria.

Por otro lado, a nivel de aplicación se podría implementar lógica que controle las actualizaciones en todas las copias de un dato o utilizar microservicios o capas intermedias que gestionen una mejor coherencia entre las versiones duplicadas del mismo dato.

- **¿Qué problemas puede provocar un mal diseño de las claves de partición respecto a la consistencia y disponibilidad de los datos?**

Un mal diseño de las claves de una partición puede ocasionar varios problemas como los siguientes:

Se pueden producir Hotspots si muchas operaciones se dirigen a una sola partición (por ejemplo, por una clave muy repetida), un solo nodo soportará toda la carga, afectando el rendimiento y la disponibilidad.

También pueden ocurrir Desbalances de carga si algunas particiones concentran muchos datos y otras muy pocos, los nodos del clúster no se aprovechan de forma equilibrada.

Puede afectar también a la consistencia ya que con claves mal distribuidas y niveles de consistencia bajos podrían devolverse datos desactualizados al usuario.

7. Conclusiones obtenidas: ¿Crees que tu modelo de datos encaja mejor en un RDBMS o se adapta mejor a Cassandra?

- **¿Qué cambios has tenido que hacer respecto a un diseño relacional tradicional? ¿Cómo te ha condicionado pensar en las consultas desde el principio?**

Este diseño ha variado bastante respecto a las anteriores practicas ya que antes necesitábamos simplemente las tablas principales, pero ahora algunas de esas tablas principales han sido duplicadas según su función, ya sea para la clave primaria (el nombre o el id) o para hacer tablas extra para no hacer uso de JOIN en las consultas.

Esto ha sido condicionado pensando en las consultas, ya que hemos tenido que crear las tablas conforme realizábamos las consultas, así que esto ha podido derivar a que igual la base de datos no sea la más óptima y sencilla posible.

- **¿Qué ventajas e inconvenientes has observado en el uso de Cassandra para el caso de uso planteado?**

Algunas de las ventajas que hemos encontrado en Cassandra son su alta escalabilidad horizontal y vertical, su rendimiento en tiempo real, sus escrituras rápidas y su modelo de consistencia configurable

Hay que añadir que no tiene un punto único de fallo (arquitectura peer-to-peer) y cuenta con un soporte multi datacenter

Por otro lado, las desventajas que hemos podido observar son que gestiona los nodos y las particiones complejamente, su modelo de datos es limitado ya que no soporta JOINS complejos y las consultas son complicadas de implementar.

Pero lo que más hemos notado es que necesita un monitoreo constante, necesita mucho consumo de almacenamiento y no es adecuado para todos los escenarios de datos

8. Conclusiones finales

Como conclusiones finales y según la experiencia que hemos tenido personalmente Cassandra no nos ha gustado mucho desde un principio ya que para su empleo necesitábamos Docker, un programa el cual consume mucha RAM, esto por ejemplo no ha permitido a Mario Recio poder hacer las pruebas de inserción grandes y algunas de las consultas ya que su ordenador no tenía tanta RAM pero las hemos podido completar ambos en el ordenador de Álvaro Pérez.

No creemos que sea el gestor más óptimo para nuestra base de datos ya que tiene bastantes variables, y esto produciría que si queremos hacer ciertas consultas haga que tengamos que crear demasiadas tablas, lo cuál en cuanto a la eficiencia de ejecución de las consultas es muy buena y rápida ya que se adapta la base de datos a los tipos de consultas que quieres hacer para obtener esos datos de manera más fácil a la hora de consultarlo, pero hace que sea mucho más complejo de gestionar para insertarlos. En cuanto a la inserción es el más lento en cuanto a ejecución de los tres SGBD vistos.

Finalmente, aunque no haya sido mucho de nuestro agrado este SGBD, nos quedamos con que Cassandra nos ha permitido aprender otra forma nueva de distribuir bases de Datos y comprender otro tipo de planteamiento más focalizado a las consultas de una forma más directa.

9. Anexo

Las consultas ya vienen en la memoria así que no las subimos como archivo, y tema dataset al igual que en la práctica dos utilizamos el mismo para los tres tipos (bajo, medio y alto).